

Feed Forward Neural Network (AI)

Aim

To implement a Feed Forward Neural Network with one hidden layer, trained using backpropagation, to learn the non-linearly separable XOR function.

Objectives

- To understand the architecture of a feed forward neural network.
- To implement forward propagation using the sigmoid activation function.
- To implement backpropagation to update weights and biases.
- To train the network until it correctly learns the XOR function.

Algorithm

- 1 Start.
- 2 Initialize the XOR input-output dataset and random weights/biases.
- 3 Perform forward propagation: compute hidden layer output and final output.
- 4 Compute the error between the predicted and actual output.
- 5 Perform backward propagation: compute gradients using the derivative of the sigmoid function.
- 6 Update the weights and biases using the learning rate and computed gradients.
- 7 Repeat forward and backward propagation for a fixed number of epochs.
- 8 Periodically display the training loss.
- 9 Test the trained network on all XOR input combinations.
- 10 Stop.

Source Code (Python)

```
import numpy as np
np.random.seed(1)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
# XOR dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])
input_neurons, hidden_neurons, output_neurons = 2, 4, 1
w1 = np.random.uniform(size=(input_neurons, hidden_neurons))
b1 = np.random.uniform(size=(1, hidden_neurons))
w2 = np.random.uniform(size=(hidden_neurons, output_neurons))
b2 = np.random.uniform(size=(1, output_neurons))
lr = 0.5
epochs = 10000
for epoch in range(epochs):
    # Forward pass
    hidden_input = np.dot(X, w1) + b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, w2) + b2
```

```

final_output = sigmoid(final_input)
# Backward pass
error = y - final_output
d_output = error * sigmoid_derivative(final_output)
error_hidden = d_output.dot(w2.T)
d_hidden = error_hidden * sigmoid_derivative(hidden_output)
w2 += hidden_output.T.dot(d_output) * lr
b2 += np.sum(d_output, axis=0, keepdims=True) * lr
w1 += X.T.dot(d_hidden) * lr
b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr
if epoch % 2000 == 0:
    loss = np.mean(np.square(error))
    print(f"Epoch {epoch:5d} Loss: {loss:.6f}")
print("\nFinal predictions after training (XOR problem):")
for i in range(len(X)):
    print(f"Input: {X[i]} -> Predicted: {final_output[i][0]:.4f} Rounded:
{round(final_output[i][0])} Expected: {y[i][0]}")

```

Output

```

Epoch      0 Loss: 0.367371
Epoch  2000 Loss: 0.005436
Epoch  4000 Loss: 0.000965
Epoch  6000 Loss: 0.000475
Epoch  8000 Loss: 0.000307
Final predictions after training (XOR problem):
Input: [0 0] -> Predicted: 0.0123 Rounded: 0 Expected: 0
Input: [0 1] -> Predicted: 0.9881 Rounded: 1 Expected: 1
Input: [1 0] -> Predicted: 0.9823 Rounded: 1 Expected: 1
Input: [1 1] -> Predicted: 0.0172 Rounded: 0 Expected: 0

```

Result

The Feed Forward Neural Network was successfully trained using backpropagation and learned to correctly predict the XOR function for all four input combinations.